

Assignment 7: Introduction to RLHF

Overview

In this assignment, you will learn to implement a VERL (Volcano Engine Reinforcement Learning) like framework, a flexible and efficient reinforcement learning framework designed for large language models. You'll implement a basic RLHF (Reinforcement Learning from Human Feedback) pipeline to fine-tune a small language model for harmless and helpful responses.

Learning Objectives

After completing this assignment, you will be able to:

- Understand the basic concepts of RLHF and its application to LLMs
- Set up and configure the VERL like framework
- Implement a simple reward model for evaluating LLM outputs
- Use VERL like framework to fine-tune a language model
- Evaluate the performance improvements after RLHF training

Background Reading

Before starting the implementation, familiarize yourself with the core concepts:

Essential Reading: - [VERL Documentation](#) - Framework overview and API reference - [HybridFlow Paper](#) - The research foundation behind VERL - [RLHF Tutorial](#) - Comprehensive introduction to RLHF methodology - [PPO Algorithm Explained](#) - Understanding the RL algorithm

Key Concepts: - **RLHF Pipeline:** Human preference data → Reward model training → Policy optimization with PPO - **VERL's Hybrid Architecture:** Separation of generation (inference) and training phases for scalability - **PPO Algorithm:** Policy gradient method with clipping for stable training - **Reward Model:** Neural network trained to predict human preferences

Environment Setup

Step 1: Clone the Assignment Repository

```
git clone https://github.com/llmsystem/llmsys_hw7.git
cd llmsys_hw7
```

Step 2: Create a Virtual Environment

```
python -m venv llm_sys_hw7  
source llm_sys_hw7/bin/activate
```

Step 3: Install Dependencies

```
# Install dependencies  
pip install -r requirements.txt
```

Training Data

Data Sources

This assignment uses the **Anthropic/hh-rlhf** dataset from Hugging Face, which contains real human preference data for helpfulness and harmlessness. This is the same dataset used in many RLHF research papers.

Data Loading and Preparation

The training data is automatically downloaded and prepared when you first run the training scripts. You can also prepare it manually:

```
# Download and prepare the dataset  
python scripts/prepare_data.py --dataset Anthropic/hh-rlhf --output_dir data --  
max_samples 10000
```

Problems

Problem 1: Implementing a Reward Model (40 points)

1.1 Complete the loss implementation in `src/reward_model.py`:

In this problem, you

- Implement the ranking loss for the reward training in `compute_loss`.
- The model is based on a pre-trained transformer (DistilBERT)

1.2 Train your reward model using the provided preference data:

```
PYTHONPATH=$(pwd) python scripts/train_reward_model.py
```

Problem 2: RLHF Training with VERL like framework (40 points)

2.1 Complete the RLHF trainer implementation in `src/rlhf_trainer.py`:

- Implement the `VERLTrainer` class

2.2 Run the RLHF training process:

```
PYTHONPATH=$(pwd) python scripts/run_rlhf.py --model_name gpt2
```

Problem 3: Evaluation and Analysis (20 points)

3.1 Run comprehensive evaluation:

```
PYTHONPATH=$(pwd) python scripts/evaluate.py --base_model gpt2 --rlhf_model  
outputs/rlhf_model
```

You may notice gibberish generations receiving high rewards. This is expected, as the reward model is neural model and the policy model learns to exploit its loopholes.

Testing

Run the provided tests to verify your implementation:

```
python -m pytest tests/ -v
```

Submission Instructions

- 1. Code Submission:**
2. Ensure all code runs without errors
3. Include all required output files
4. Test your implementation with `python -m pytest tests/`
- 5. Create Submission Archive:**

```
# Remove large model files but keep small checkpoints  
find outputs/ -name "*.pt" -size +500M -delete
```

```
# Create submission zip  
zip -r assignment7_[your_andrew_id].zip . -x "*.git*" "__pycache__*" "*.pyc"
```

Grading Criteria

Problem 1: pass the pytest and achieve at least 60% validation acc with reward model

Problem 2: observe reasonable improvement of reward after RLHF training (e.g., -0.5 to +0.5) in the `rlhf_training_curves.png`

Problem 3: see clearly different reward distributions of model before and after RLHF in figure `reward_comparison.png` and also make sure to upload the model checkpoints of best reward model and best rlhf model so TA can reproduce the evaluation.